

DEPARTMENT OF COMPUTER ENGINEERING

Course name: LLM

Course Code: CE453

Sem: VI A.Y. 2026-27

DETAILS	
Name	Anurag Akhilesh Singh
UID no.	2023300231
Batch	2

EXPERIMENT NO. 2	
AIM :	To study how selected hyperparameters affect the performance of a task-specific large language model fine-tuned using Parameter-Efficient Fine-Tuning (PEFT) with Low-Rank Adaptation (LoRA), while maintaining controlled experimental conditions. The practical will: • use PEFT and LoRA • configure a Qwen language model for LoRA fine-tuning • define a baseline training configuration • run multiple controlled configurations • compare training behaviour and generated responses • select a suitable configuration based on evidence
PROBLEM STATEMENT:	Context: A fine-tuning pipeline can operate with many possible settings. A learning rate or LoRA rank that works well for one task may not work well for another. Research Question: Which combination of learning rate and LoRA rank produces the best balance of: 1. Task-specific response quality 2. Training loss convergence 3. Training time efficiency 4. Parameter efficiency 5. Overfitting risk Scope: This experiment investigates this question for customer support Q&A using a small pretrained model (0.5B parameters) with a domain-specific dataset (Bitext customer support examples).

DEPARTMENT OF COMPUTER ENGINEERING

Course name: LLM

Course Code: CE453

Sem: VI A.Y. 2026-27

THEORY :

Large Language Models (LLMs)

Large language models are neural networks trained to predict the next token in a sequence, using transformer architecture (Vaswani et al., 2017). Models like Qwen2.5 are trained on diverse text corpora to learn general language patterns.

Characteristics:

- **Causal Language Modeling:** Predicts next token given preceding context
- **Pretrained Knowledge:** Contains general world knowledge and linguistic patterns
- **Scale:** Model size ranges from 7M to 70B+ parameters

Instruction Tuning

Instruction-tuned models (like Qwen2.5-0.5B-Instruct) are fine-tuned on instruction-following examples to align with user intent. The model learns to follow structured instructions rather than merely complete text.

Format: User query → Model response (conversational interface)

Advantage: Better alignment with human expectations compared to base causal models

Parameter-Efficient Fine-Tuning (PEFT)

Full fine-tuning updates all model parameters, requiring:

- Gradient storage for all parameters (substantial memory)
- Optimizer states (momentum, variance) for all parameters
- Compute for backpropagation through the entire model

PEFT Solution: Keep most pretrained weights frozen; train only a small adapter module.

Advantage: 10–1000× reduction in trainable parameters, enabling fine-tuning on consumer hardware

LoRA (Low-Rank Adaptation)

DEPARTMENT OF COMPUTER ENGINEERING

Course name: LLM

Course Code: CE453

Sem: VI A.Y. 2026-27

LoRA represents the weight update as a product of two low-rank matrices:

Mathematical Formulation:

Original weights: $W \in \mathbb{R}^{(d_{\text{out}} \times d_{\text{in}})}$

LoRA update: $\Delta W \approx B \times A$

where:

$A \in \mathbb{R}^{(r \times d_{\text{in}})}$ [low-rank matrix, rank r]

$B \in \mathbb{R}^{(d_{\text{out}} \times r)}$ [low-rank matrix, rank r]

Effective weights: $W' = W + \alpha/r \times B \times A$

Key Parameters:

- **r (Rank):** Controls the expressiveness of the adapter (capacity trade-off)
- **α (Alpha):** Scaling factor for the update magnitude
- **Target Modules:** Which layers receive LoRA adapters (e.g., q_{proj} , v_{proj})

Advantages:

- Low-rank constraint limits overfitting
- Efficient: only $r \times (d_{\text{in}} + d_{\text{out}})$ parameters trained instead of $d_{\text{in}} \times d_{\text{out}}$
- Additive formulation: no architectural changes to base model

4.5 Learning Rate in Optimization

The learning rate (LR) controls the magnitude of optimizer updates:

$$w_{\text{new}} = w_{\text{old}} - \text{LR} \times \nabla \text{loss}$$

Effects:

- **Too low:** Slow convergence, risk of underfitting
- **Appropriate:** Efficient convergence to good solution
- **Too high:** Unstable training, oscillations, divergence

For fine-tuning, typical learning rates are $1e-5$ to $1e-3$, with smaller rates for larger models and larger rates for smaller, task-specific adaptation

DEPARTMENT OF COMPUTER ENGINEERING

Course name: LLM

Course Code: CE453

Sem: VI A.Y. 2026-27

CODE :	 2023300231_LLM_EXP_2_ANURAG.ipynb																				
OBSERVATIONS:	The experiment was performed using Qwen2.5-0.5B-Instruct with Parameter-Efficient Fine-Tuning (PEFT) using LoRA. The same base model, dataset, preprocessing pipeline, tokenizer/chat template, LoRA target modules and evaluation procedure were maintained across the experiments so that the effect of the selected hyperparameters could be compared fairly. The experiment specifically studied the effect of learning rate and LoRA rank (r). This follows the controlled experimental approach recommended in the experiment specification. The LoRA configuration used q_proj and v_proj as target modules, with lora_alpha = 16, lora_dropout = 0.05, bias = none, and CAUSAL_LM as the task type. LoRA rank was varied as the experimental capacity parameter, while the other LoRA settings were kept fixed. A higher rank provides greater adapter capacity but also increases the number of trainable parameters. Table 1: Hyperparameter Configurations <table><tr><th>Run</th><th>Learning Rate</th><th>Epochs</th><th>LoRA Rank (r)</th><th>Experimental Purpose</th></tr><tr><td>Run A</td><td>1×10^{-4}</td><td>2</td><td>4</td><td>Baseline configuration</td></tr><tr><td>Run B</td><td>2×10^{-4}</td><td>2</td><td>4</td><td>Study effect of learning rate</td></tr><tr><td>Run C</td><td>2×10^{-4}</td><td>2</td><td>8</td><td>Study effect of LoRA rank</td></tr></table> Run A and Run B differ only in learning rate, allowing the effect of learning rate to be observed. Run B and Run C differ only in LoRA rank, allowing the effect of adapter capacity to be examined. This controlled structure is consistent with the recommended Run A → Run B → Run C methodology.	Run	Learning Rate	Epochs	LoRA Rank (r)	Experimental Purpose	Run A	1×10^{-4}	2	4	Baseline configuration	Run B	2×10^{-4}	2	4	Study effect of learning rate	Run C	2×10^{-4}	2	8	Study effect of LoRA rank
Run	Learning Rate	Epochs	LoRA Rank (r)	Experimental Purpose																	
Run A	1×10^{-4}	2	4	Baseline configuration																	
Run B	2×10^{-4}	2	4	Study effect of learning rate																	
Run C	2×10^{-4}	2	8	Study effect of LoRA rank																	

DEPARTMENT OF COMPUTER ENGINEERING

Course name: LLM

Course Code: CE453

Sem: VI A.Y. 2026-27

Table 2: Final Hyperparameter Tuning Results

Run	Learning Rate	Epoch	LoRA Rank	Trainable Parameters	Trainable %	Final Loss	Training Time
Run A	0.0001	2	4	270,336	0.0547%	1.037825	6.98 min
Run B	0.0002	2	4	270,336	0.0547%	0.947160	6.90 min
Run C	0.0002	2	8	540,672	0.1093%	0.920775	6.89 min

Observation 1 : Effect of Learning Rate :

Run A and Run B use the same LoRA rank of 4 and the same number of epochs. The only changed hyperparameter is the learning rate, which increases from 1×10^{-4} to 2×10^{-4} .

The final loss decreased from **1.037825 in Run A to 0.947160 in Run B**, representing an approximately **8.74% reduction** in final loss.

Therefore, under the conditions of this experiment, the higher learning rate resulted in better convergence according to the recorded final training loss. This observation is specific to the tested configurations and does not imply that a higher learning rate is universally better. The experiment specification notes that excessively high learning rates can cause unstable or fluctuating training, while excessively low learning rates can result in slow learning or under-adaptation.

Observation 2 : Effect of LoRA Rank

Run B and Run C use the same learning rate of 2×10^{-4} and the same number of epochs. The LoRA rank is increased from **4 to 8**.

The final loss decreased from **0.947160 to 0.920775**, representing an approximately **2.79% reduction**.

At the same time, the number of trainable parameters increased from **270,336**

DEPARTMENT OF COMPUTER ENGINEERING

Course name: LLM

Course Code: CE453

Sem: VI A.Y. 2026-27

to **540,672**, which is exactly a **2× increase**. This demonstrates the trade-off between LoRA capacity and parameter efficiency. A higher rank provides greater capacity for the low-rank adaptation but requires more trainable parameters.

Observation 3 : Overall Loss Improvement

Comparing Run A with Run C, the final loss decreased from **1.037825** to **0.920775**, corresponding to an overall reduction of approximately **11.30%**.

Run C therefore recorded the **lowest final loss among the three tested configurations**.

However, the lower loss was obtained with twice the number of trainable parameters compared with the rank-4 configurations. Hence, the results demonstrate a trade-off between adaptation capacity and parameter efficiency rather than proving that a larger LoRA rank is always preferable.

Table 3: Parameter Efficiency Comparison

Configuration	LoRA Rank	Trainable Parameters	Trainable Percentage
Run A	4	270,336	0.0547%
Run B	4	270,336	0.0547%
Run C	8	540,672	0.1093%

Observation :

LoRA significantly reduces the number of parameters that need to be updated during fine-tuning because the pretrained model remains largely frozen and only the LoRA adapter parameters are trained. Increasing the rank from 4 to 8 doubled the trainable parameter count from **270,336 to 540,672**, while still keeping the trainable portion very small relative to the complete model. This demonstrates the parameter-efficient nature of PEFT/LoRA while also showing that increasing adapter rank increases the available adaptation capacity.

DEPARTMENT OF COMPUTER ENGINEERING

Course name: LLM

Course Code: CE453

Sem: VI A.Y. 2026-27

Table 4: Comparative Hyperparameter Analysis :

Comparison	Parameter Changed	Observed Result	Interpretation
Run A → Run B	Learning rate: 1e-4 → 2e-4	Loss: 1.037825 → 0.947160	Higher tested learning rate produced lower final loss
Run B → Run C	LoRA rank: 4 → 8	Loss: 0.947160 → 0.920775	Higher tested rank produced further loss reduction
Run A → Run C	LR and rank changed through controlled sequence	Loss reduced by ≈11.30%	Run C achieved the lowest recorded loss
Run B → Run C	Trainable parameters doubled	270,336 → 540,672	Higher rank increased adaptation capacity and parameter cost
All runs	Training time	≈6.9 min	No substantial training-time difference observed

Table 6: Overall Experimental Observations

Aspect	Observation
Base Model	Qwen2.5-0.5B-Instruct was kept fixed across the experiments.
Fine-Tuning Method	PEFT using LoRA was used for parameter-efficient adaptation.
Target Modules	q_proj and v_proj were used as LoRA target modules.
LoRA Configuration	alpha = 16, dropout = 0.05, bias = none, causal LM task.

DEPARTMENT OF COMPUTER ENGINEERING

Course name: LLM

Course Code: CE453

Sem: VI A.Y. 2026-27

	Learning Rate Effect	Increasing LR from 1e-4 to 2e-4 reduced final loss from 1.037825 to 0.947160.
	LoRA Rank Effect	Increasing rank from 4 to 8 reduced final loss from 0.947160 to 0.920775.
	Parameter Efficiency	Rank 4 used 270,336 trainable parameters; rank 8 used 540,672.
	Lowest Recorded Loss	Run C achieved the lowest final loss of 0.920775.
	Training Efficiency	All runs required approximately 6.9 minutes.
	Task-Specific Evaluation	The same unseen prompt should be used across configurations to assess practical response quality.
CONCLUSION:	I implemented the LoRA fine-tuning experiment using Qwen2.5-0.5B-Instruct and compared three controlled configurations by varying the learning rate and LoRA rank. I observed that increasing the learning rate from 1e-4 to 2e-4 reduced the final loss from 1.037825 to 0.947160, while increasing the LoRA rank from 4 to 8 further reduced it to 0.920775. I also observed that the higher rank doubled the trainable parameters from 270,336 to 540,672, while training time remained approximately 6.9 minutes. Among the tested configurations, Run C achieved the lowest recorded loss, demonstrating the trade-off between adaptation capacity and parameter efficiency. Overall, I found that controlled hyperparameter tuning improved the measured fine-tuning performance while retaining the parameter-efficiency benefits of LoRA.	